

Tutor Bot — Technical Report

AI Engineer Assignment · a Pipecat + OpenAI voice agent that teaches a class about natural disasters

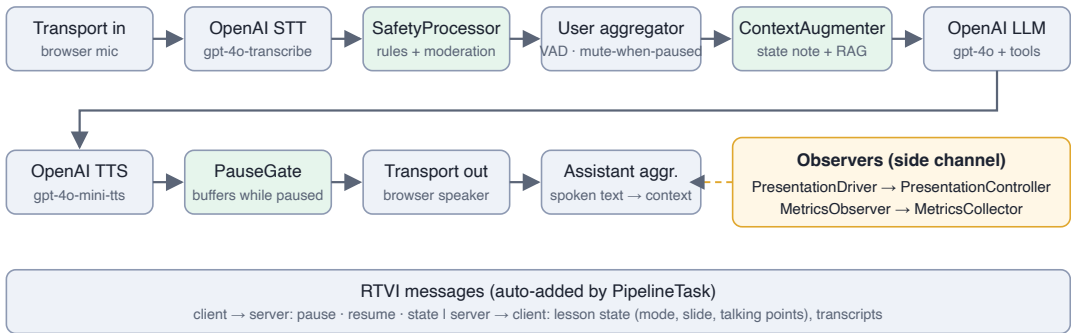
1. Overview

Tutor Bot is a real-time voice agent that presents an eight-slide lesson on natural disasters to school students (ages ~10–14), answers their questions mid-lesson, returns to the slides, and finishes with a two-way Q&A. The starting point was a prototype whose slide progression depended on silence timers and whose behaviour was unpredictable. The submission rebuilds the agent around a small, framework-free state machine, keeps every piece of business logic unit-testable without a network, and uses Pipecat as the only framework and OpenAI as the only external service.

Item	Value
Framework	Pipecat 1.11 (Python 3.11), FastAPI WebSocket transport, Vite + TypeScript client using @pipecat-ai/client-js
External services	OpenAI only: gpt-4o-transcribe (realtime STT), gpt-4o (LLM, tool calling), gpt-4o-mini-tts (TTS), text-embedding-3-small (knowledge retrieval), optional moderation endpoint. Silero VAD and Smart Turn run locally inside Pipecat.
Tests	115 deterministic pytest tests (no network, ~5 s) + 10 LLM-as-a-judge evals
Repository layout	main.py (server), agent.py (pipeline wiring), tutor/ (all logic), tests/, evals/, knowledge/, frontend/

2. Architecture

2.1 Pipeline



Green boxes are components written for this submission; the rest are Pipecat/OpenAI services. Observers see every frame without being in the audio path, which is where the slide-progression logic lives.

2.2 Key components

Component	Responsibility
<code>tutor/controller.py</code> PresentationController	Pure state machine: modes <code>IDLE</code> → <code>PRESENTING</code> → <code>QNA</code> → <code>ENDED</code> , current slide, paused flag, whether the student interrupted mid-slide, statistics. It receives events (bot started/stopped speaking, user started/stopped, advance, pause, stall, idle) and returns <i>Actions</i> (present slide, continue slide, enter Q&A, idle prompt, end session, schedule advance). No timers, no I/O, no Pipecat imports.
<code>tutor/processors/presentation_driver.py</code> PresentationDriver (observer)	Executes the controller's actions inside Pipecat: injects slide prompts via <code>LLMMessagesAppendFrame(run_llm=True)</code> , runs the 1.2 s slide gap, an 8 s stall watchdog, the Q&A idle timer, pause/resume, graceful end (<code>stop_when_done</code>), and broadcasts state to the UI. It guards the advance with an "LLM busy / tool call pending" check so a slow generation or a tool round-trip never causes a premature slide change.
<code>tutor/prompts.py</code>	System prompt, per-slide messages, Q&A / idle / goodbye messages, and <code>build_state_note()</code> — the <code>[PRESENTER STATE]</code> note refreshed before every inference.
<code>tutor/processors/context_augmenter.py</code>	Sits between the user aggregator and the LLM. On every <code>LLMContextFrame</code> it replaces the previous state note with a fresh one and, when the trigger is a student utterance, embeds it and appends the top-3 knowledge passages.
<code>tutor/tools.py</code>	LLM tools <code>go_to_slide(slide_number topic)</code> , <code>next_slide()</code> , <code>end_session()</code> . Handlers update the controller and return the slide content as the tool result, so the LLM continues presenting in the same turn.
<code>tutor/processors/pause_gate.py</code>	Frame processor between TTS and the output transport. While paused it holds every downstream data/control frame (audio, text, TTS start/stop, LLM response end) in order; on resume it releases them. System frames and upstream frames pass through so the pipeline stays healthy.
<code>tutor/safety.py</code> + <code>processors/safety_processor.py</code>	Deterministic guard (regex rules per category + PII detectors) applied to each transcription before it reaches the aggregator. Blocked text is dropped; a fixed spoken reply is emitted via <code>TTSSpeakFrame</code> ; a one-line system note keeps the LLM context coherent. Optional OpenAI moderation layer. <code>PausedUserMuteStrategy</code> suppresses VAD/interruptions/transcriptions while paused.
<code>tutor/knowledge/</code>	Markdown chunker (by heading, paragraph-packed, content-hash ids), numpy cosine <code>VectorStore</code> with JSON persistence, incremental <code>ingest_documents()</code> , OpenAI embedder, CLI.
<code>tutor/metrics.py</code> + <code>processors/metrics_observer.py</code>	Collector fed by Pipecat <code>MetricsFrame</code> s (TTFB, processing, LLM tokens, TTS characters, STT seconds) plus our own signals (user→bot response latency, tool calls, pauses, safety blocks). Renders the console report on disconnect; <code>GET /report/latest</code> serves it to the UI.
<code>tutor/transcripts.py</code> + <code>tutor/improve/</code>	JSONL session recorder (PII-redacted) and the offline improvement loop (analyzer + LLM judge + FAQ/guidance generation).
<code>frontend/app.ts</code>	Connect / Pause / Resume / Disconnect; renders server-pushed lesson state; suspends the WebAudio <code>AudioContext</code> on pause for instant, sample-exact resume; shows the metrics report after disconnect.

2.3 Important technical decisions

- **Deterministic slide progression, not LLM-driven.** Each slide is one LLM turn. When the bot finishes speaking and 1.2 s of silence passes (and no LLM response or tool call is in flight), the controller advances. The LLM only ever receives the content of the current slide, so it cannot run ahead, and the end of the deck is a state-machine transition rather than a hope that the model calls a function. Tools exist for *student-requested* navigation only.
- **Per-inference state note.** Rather than trying to inject a message at exactly the right moment after an interruption (racy), a small note describing the exact situation is rewritten before *every* LLM call. Pipecat's assistant aggregator commits only the text that was actually spoken before an interruption, so "continue from where you stopped" is grounded in the real context.
- **Hybrid pause.** The WebSocket transport sends audio at 2× real time, so a server-only gate would leave seconds of audio already buffered in the browser. The server gate is the authority (and is unit-tested); the browser's `AudioContext.suspend()` makes the pause instant and the resume sample-exact.
- **Rules first, model second, for safety.** Regex rules are exact, explainable and testable, so they form the first layer; the system prompt handles paraphrases; the moderation endpoint is available as a third layer. Blocked content never reaches the LLM.
- **Framework-free core.** Everything in `tutor/` except `processors/` and `tools.py` has no Pipecat dependency, which is what makes 115 fast deterministic tests possible. A separate wiring test constructs the real pipeline and asserts the processor order and tool registration.
- **Recovery paths.** A stall watchdog (no activity for 8 s while presenting) re-prompts the model; a false VAD trigger that yields no transcription cannot freeze the lesson. In Q&A, idle prompts lead to a graceful goodbye and a clean pipeline shutdown so the metrics report always prints.

3. Goal coverage

Goal	Implementation	Verification
Smoothly return to topic once questions are answered	<code>controller.on_user_started_speaking()</code> marks <code>interrupted_mid_slide</code> when the student speaks while the bot is presenting. The next state note instructs: answer briefly, transition back, continue from exactly where you stopped, do not repeat. If the student spoke in the gap after a slide, the note says to answer and let the system continue. After the reply, the normal gap advances the deck. If a safety rule blocked the interruption, <code>resume_after_reply</code> makes the controller issue <i>continue slide</i> instead of advancing.	Unit: <code>test_controller.py</code> , <code>test_prompts.py</code> . Eval: <code>test_returns_to_topic_after_mid_slide_interrupt</code>
Reliably end the presentation and enter Q&A; return to any slide on request	After the last slide's turn completes, <code>advance()</code> returns <i>enter Q&A</i> ; the driver injects the Q&A prompt and stops auto-advancing. In any mode, <code>go_to_slide / next_slide</code> tools jump; the controller switches back to PRESENTING, and the deck resumes to the end and re-enters Q&A. Q&A idle prompts (25 s, twice) end with a goodbye and <code>end_session</code> .	Unit: full-deck and jump tests in <code>test_controller.py</code> , <code>test_driver.py</code> . Evals: <code>test_qna_go_back_to_slide_uses_tool</code> , <code>test_qna_finishing_calls_end_session</code> .
Pause from the frontend; resume exactly where it left off	Client sends RTVI <code>pause / resume</code> ; server pauses the controller (cancels timers), closes the <code>PauseGate</code> , mutes the student; browser suspends its <code>AudioContext</code> and mic. Resume reverses it and re-schedules the slide gap if the slide had already finished.	Unit: <code>test_pause_gate.py</code> (order preserved, system frames bypass, end-frame flush), <code>test_driver.py</code> .
Guardrails for underage users	Nine rule categories (self-harm, PII request, PII sharing, off-platform contact, sexual, dangerous how-to, violence, drugs, hate) plus phone/email/address/handle detectors; fixed child-appropriate replies; PII redaction in transcripts; strict prompt rules; optional moderation API.	Unit: 37 tests in <code>test_safety.py</code> . Evals: graphic question, s child, PII offer, off-topic redirect.
Deterministic tests for all business logic	115 tests over controller, prompts, safety, chunking/vector store (fake embedder), metrics, transcripts, analyzer, pause gate, driver timers (fake task) and real pipeline wiring.	<code>uv run pytest</code> — 115 passed, ~5 s, no network.
Eval-style (LLM-as-a-judge) tests	<code>evals/harness.py</code> replays the real system prompt, slide messages, state note and tool schemas through the OpenAI chat API; a judge model grades replies against explicit criteria. Ten scenarios cover slide fidelity, return-to-topic, Q&A tools, knowledge use and safety.	<code>uv run pytest evals -m eval</code> (requires an API key; skip otherwise).
Reliable knowledge ingestion	<code>python -m tutor.knowledge.ingest</code> : Markdown/text → heading-aware chunks → embeddings → JSON index. Incremental by content hash and model name; removed documents are dropped. Four documents (~2,400 words) ship with the project; the <code>ContextAugmenter</code> injects top-3 passages above a similarity threshold on every student question.	Unit: <code>test_knowledge.py</code> (ranking, persistence, incremental reuse). Eval: <code>test_uses_retrieved_knowledge</code> .
Metric report on console after disconnect	<code>MetricsObserver</code> dedupes frames per pipeline hop and feeds the collector; on disconnect <code>run_bot</code> prints the report and stores it for <code>GET /report/latest</code> . Includes avg/min/max/p95	Unit: <code>test_metrics.py</code> . Example output below.

latencies, tokens per model, TTS/STT usage, tool calls, events and an estimated cost.

EXAMPLE OUTPUT (format produced by `tutor.metrics.render_report`; values are illustrative)

```
=====
                        SESSION METRICS REPORT
=====
Duration: 318.0s   User turns: 4   Bot turns: 12
=====
LATENCY (seconds)      avg    min    max    p95    n
  TTFB OpenAILLM        0.669  0.550  0.800  0.770  12
  TTFB OpenAITTS        0.315  0.280  0.360  0.350  12
  Processing OpenAILLM  2.233  1.800  2.700  2.600  12
  User -> bot response   1.115  0.980  1.310  1.310  4
=====
LLM TOKENS              calls  prompt completion  total
  gpt-4o                 12    33600         1533    35133
=====
TTS: 12 requests, 6456 characters   STT: 312.4s of audio
=====
TOOL CALLS: end_session=1, go_to_slide=1
=====
EVENTS: knowledge_hits=6, pause=1, safety_block=1
=====
Estimated cost: $0.2274 USD (list prices; approximate)
=====
```

4. Open-ended goal: learning from the agent's own transcripts

Every session is recorded to `data/transcripts/<session>.jsonl`: student and bot turns (with slide, mode and whether the bot was interrupted), controller actions, pauses, safety blocks, retrieval hits, and the final metrics.

`python -m tutor.improve` turns that history into concrete improvements:

1. **Deterministic analysis** (`tutor/improve/analyzer.py`, unit-tested): question/answer pairs per slide, interruptions, stalls, safety categories, answers that sounded uncertain, per-session cost and duration.
2. **Judging**: a model scores each past answer on accuracy, age-appropriateness, concision and staying on topic, and proposes a better answer where a score is low.
3. **Knowledge**: the questions students actually asked are grouped into `knowledge/learned_faq.md` with correct, kid-friendly answers, then ingested into the vector index — so the next class gets grounded answers to the questions this class asked.
4. **Coaching notes**: up to eight actionable bullets (e.g. "on slide 4, students often ask what a tsunami is — mention it briefly") are written to `data/learned/presenter_guidance.md` and appended to the system prompt on the next start.
5. **Regression check**: the eval suite runs against the updated prompt, so a learned note that hurts behaviour is caught before it reaches students.

The loop is offline and reviewable: a teacher can read and edit the FAQ and coaching notes before they are used.

5. Status: completed, limitations, next month

What was completed in this submission. All eight goals and the open-ended goal are implemented and committed: deterministic presentation control with Q&A and jump-back, hybrid pause/resume, layered safety, incremental knowledge ingestion with automatic retrieval, a metrics report, JSONL transcripts with an improvement loop, a 115-test deterministic suite, a 10-scenario judge-based eval suite, and a rebuilt frontend that shows lesson state and controls pause. The real Pipecat pipeline is constructed in a test to verify processor order and tool registration.

Current limitations. (1) Development was done without OpenAI API credit, so the end-to-end voice session, the judge evals and the LLM step of the improvement loop have not yet been exercised live; the code paths are unit-tested and the pipeline construction is verified, but live pacing values (slide gap, stall timeout, VAD thresholds) may need tuning against real audio. (2) Return-to-topic quality still depends on the model

honouring the state note; the eval suite exists to measure it but has not produced numbers yet. (3) Safety rules are regex-based and will miss paraphrases; the prompt and optional moderation layer cover the gap but are probabilistic. (4) The knowledge base is small and English-only. (5) A single student voice is assumed; several children talking over each other is handled only by VAD, not by speaker identification.

With one additional month. First, run the live pipeline daily against a bank of recorded classroom audio and tune VAD, smart-turn and pacing from measured user→bot latency; wire the evals and the improvement loop into CI so every prompt change is graded before merge. Second, slide visuals: sync an actual slide deck (images and captions) in the UI to the spoken talking points, with per-slide progress and a teacher dashboard for pause, skip and "raise hand" queueing. Third, richer knowledge: ingest PDFs and teacher-uploaded material at runtime, add citation of sources in answers, and evaluate retrieval precision on a labelled question set. Fourth, safety depth: a fine-tuned classifier trained on the transcript corpus to replace regex, a per-class "topics to avoid" list, and an escalation channel that flags safety events to the teacher in real time. Fifth, robustness and cost: reconnect-and-resume on network loss, a smaller model for routine slide narration with gpt-4o reserved for questions, prompt caching, and OpenTelemetry traces (Pipecat supports them) feeding the metrics report.

6. Running the project

```
uv sync                                # Python 3.11 venv with all dependencies
cp .env.example .env                  # add OPENAI_API_KEY
uv run python -m tutor.knowledge.ingest # build data/knowledge_index.json (once)
uv run python main.py                 # agent on :7860
cd frontend && yarn install && yarn dev  # UI on :5173
uv run pytest                         # deterministic tests
uv run pytest evals -m eval            # LLM-as-a-judge evals
uv run python -m tutor.improve --ingest # learn from transcripts
```

See README.md for details, configuration variables and the project layout.